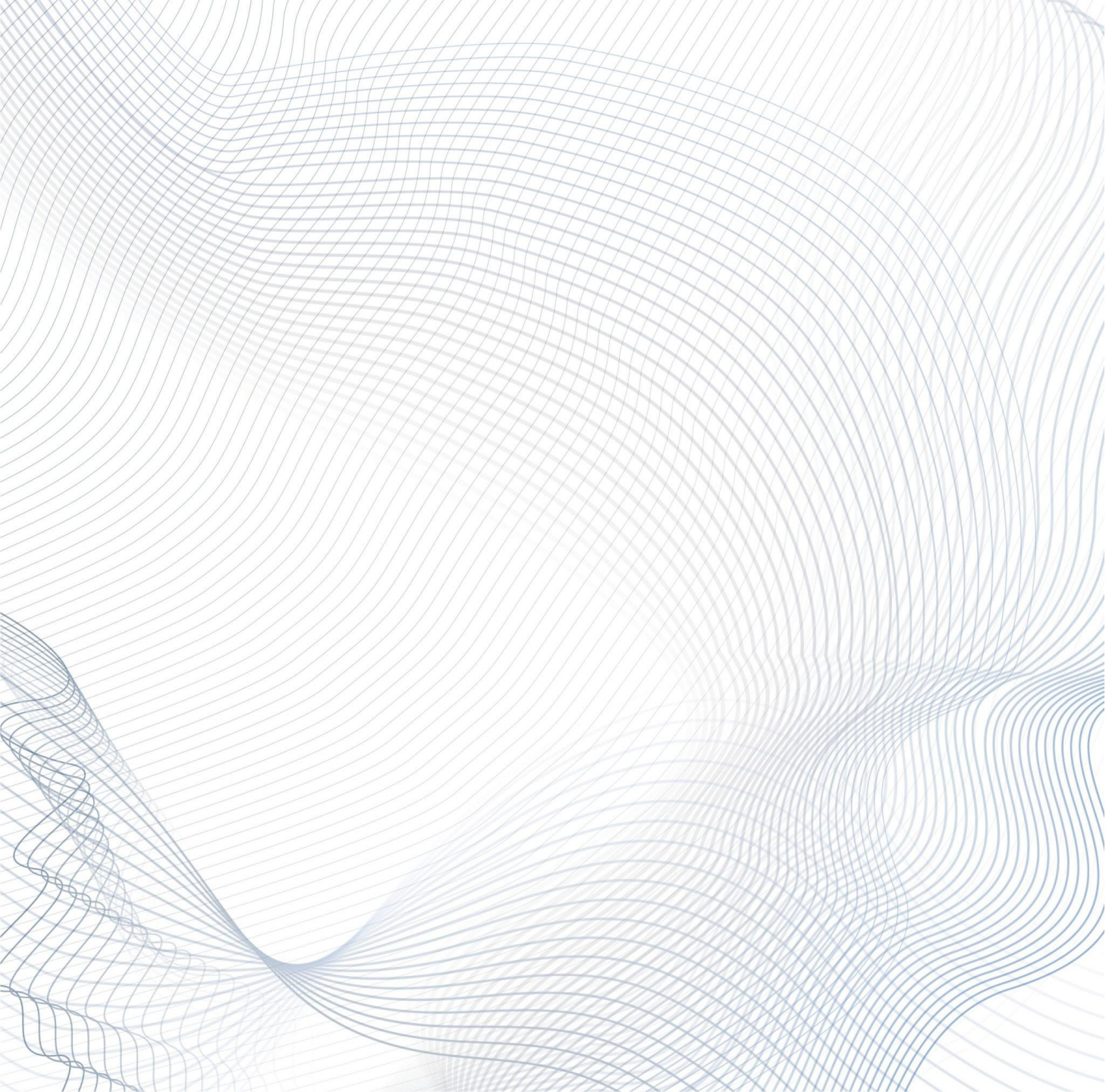


# Groth16

NITHESH

# What is Groth16?

- a pairing-based zk-SNARK for arithmetic circuits
- Proves that a secret witness satisfies a computation without revealing the witness
- Converts the computation into a Quadratic Arithmetic Program (QAP)
- Uses elliptic-curve pairings to verify the proof efficiently
- Proof size is constant: 2 elements in $\boldsymbol{G_1}$ and 1 element in $\boldsymbol{G_2}$
- Verification only requires a small number of pairings

How it works is:

# Notations

Groth16 works with three groups: $\boldsymbol{G_1}$ , $\boldsymbol{G_2}$ , and $\boldsymbol{G_T}$

$$[a]_1 = g^a \in G_1, [b]_2 = h^b \in G_2 \ [c]_T = e(g,h)^c \in G_T$$

The pairing map is: $G_1 \times G_2 \rightarrow G_T$

$g$ is a generator for $\boldsymbol{G_1}$ , $h$ is a generator for $\boldsymbol{G_2}$ , and $e(g,h)$ is a generator for $\boldsymbol{G_T}$

Properties,

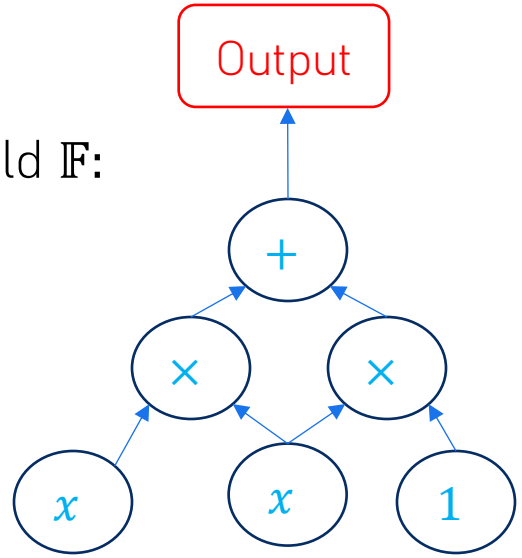
1) $e([a]_1, [b]_2) = [ab]_T$
2) $[a]_1 + [b]_1 = [a+b]_1$

# Arithmetic circuit

- Groth16 cannot prove normal code directly
- The computation is first written as an arithmetic circuit over a finite field $\mathbb{F}$:

$$+, \quad \times$$

- Each gate becomes a mathematical constraint



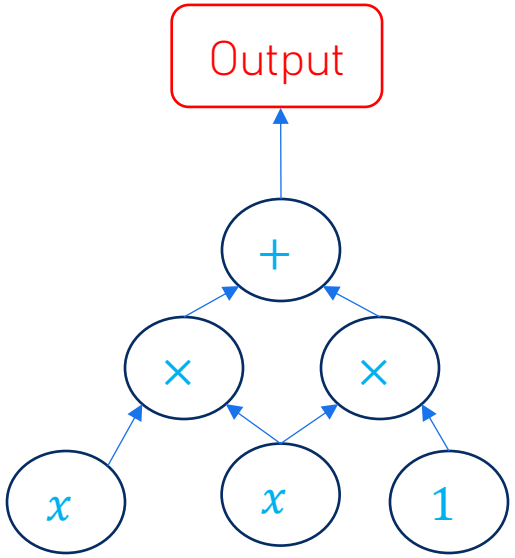

Example:

$$y = x^2 + x = x \cdot x + x \cdot 1$$

$$Let\ t = x \cdot x, then\ y = t + x$$

# R1CS

- a way to represent an arithmetic circuit as algebraic constraints. Each constraint is written as:
- $(A_q \cdot a)\,(B_q \cdot a) = C_q \cdot a$ where
- $a = (a_0, a_1, \ldots, a_m)$ *is the vector of wire values*, here $a_0 = 1$
- $A_q, B_q, C_q$ *coefficient vectors that define the* $q-th$ *constraint*
- Each constraint checks one multiplication relation



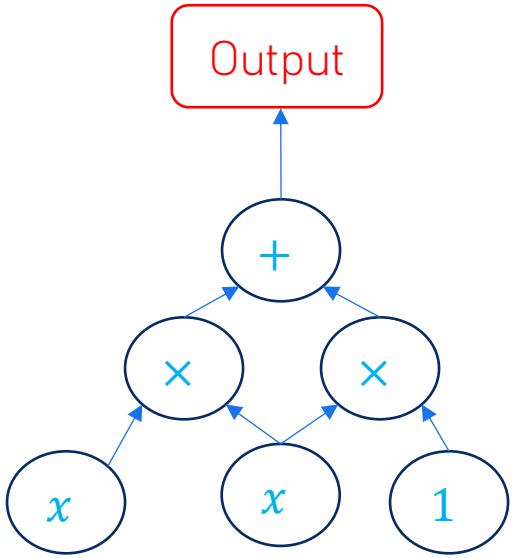

The general form:

$$\left(\sum_{i=0}^{m} a_i u_{i,q}\right)\left(\sum_{i=0}^{m} a_i v_{i,q}\right) = \sum_{i=0}^{m} a_i w_{i,q}$$

*where* $u_{i,q}, v_{i,q}, w_{i,q}$ are constants in $\mathbb{F}$ and $a_0 = 1, a_1, \ldots, a_m \in \mathbb{F}$

$$A_q = (u_{0,q}, u_{1,q}, \ldots, u_{m,q}),\ B_q = (v_{0,q}, v_{1,q}, \ldots, v_{m,q}),\ C_q = (w_{0,q}, w_{1,q}, \ldots, w_{m,q})$$

# R1CS




Example:

$$y = x^2 + x, Let\ t = x \cdot x, then\ y = t + x$$

This circuit has two constraints:

$$let\ a = (a_0, a_1, a_2, a_3) = (1, y, x, t)$$
$$such\ that\ (a_2)(a_2) = a_3\ and\ (a_3{+}a_2)(a_0) = a_1$$
$$we\ can\ have\ A_1 = (0,0,1,0), B_1 = (0,0,1,0), C_1 = (0,0,0,1)$$
$$A_2 = (0,0,1,1), B_2 = (1,0,0,0), C_2 = (0,1,0,0)$$

Constraint 1:

$$(A_1 \cdot a)\ (B_1 \cdot a) = C_1 \cdot a \rightarrow x \cdot x = t$$

Constraint 2:

$$(A_2 \cdot a)\ (B_2 \cdot a) = C_2 \cdot a \rightarrow (t + x) \cdot 1 = y$$

# QAP

- Converts R1CS constraints into polynomials so that all constraints can be checked together.
- For each *wire* $a_i$ *we construct three polynomials* $u_i(X), v_i(X), w_i(X)$.

where:

$$u_i(r_q) = u_{i,q}, \qquad v_i(r_q) = v_{i,q}, \qquad w_i(r_q) = w_{i,q}$$

Combining them:

$$U(X) = \sum_{i=0}^{m} a_i u_i(X),\ V(X) = \sum_{i=0}^{m} a_i v_i(X),\ \ W(X) = \sum_{i=0}^{m} a_i w_i(X)$$

- At each constraint point ($r_q$), the R1CS condition becomes:

$$\forall q, U(r_q)V(r_q) = W(r_q) \ \Rightarrow\ U(X)V(X) - W(X)\ vanishes\ on\ all\ r_q$$

# QAP

Since $U(X)V(X) - W(X)$ $vanishes\ on\ all\ r_1, \dots, r_n$

We can construct a vanishing polynomial $t(X)$

$$t(X) = \prod_{q=1}^{n}(X - r_q)\ \ so\ \forall r_q\ t(r_q) = 0$$

$$\text{So } t(X)\ divides\ U(X)V(X) - W(X)\ , there\ exists\ a\ polynomial\ h(X)\ of\ degree\ at\ most\ n-2$$

Thus,

$$U(X)V(X) = W(X) + h(X)t(X)$$

# QAP example

- From R1CS, choose $r_1 = 1, r_2 = 2$ , $y = x^2 + x$ $let$ $x = 3$. a $= (1, y, x, t) = (1,12,3,9)$

We have two constraints $(a_2)(a_2) = a_3$ $and$ $(a_3+a_2)(a_0) = a_1$

Then the QAP polynomials $U(X), V(X), W(X)$ $become$:

$$U(X) = a_2 u_2(X) + a_3 u_3(X) = a_2 + a_3(X-1)$$
$$V(X) = a_0\, v_0(X) + a_2 v_2(X) = a_0(X-1) + a_2(2-X)$$
$$W(X) = a_1 w_1(X) + a_3 w_3(X) = a_1(X-1) + a_3(2-X)$$

Vanishing polynomial:

$$t(X) = (X-1)(X-2)$$
$$U(X)V(X) - W(X) = -18(X-1)(X-2)$$

Thus we can get a polynomial $h(X)$:

$$U(X)V(X) - W(X) = h(X)t(X)$$
$$\therefore h(X) = -18$$

# NILP

An intermediate algebraic proof model used before the pairing-based construction

Setup:

produces a vector of field elements: $\sigma \in \mathbb{F}^{m}$

Prover:

creates a proof matrix $\Pi \, where \, \Pi \in \mathbb{F}^{k \times m}$,

Computes the proof as a linear combination of setup values : $\pi = \Pi\sigma$

Verifier:

Runs a verifier test $t \leftarrow Test(R, \phi)$ where $R$ is the public description of the circuit and $\phi$ is the public statement

Accepts only if $t(\sigma, \pi) = 0$

# NILP for QAP

Recall: $U(X)V(X) = W(X) + h(X)t(X)$

$\underline{(\sigma, \tau) \leftarrow Setup:}$

Samples $\alpha, \beta, \gamma, \delta, x \leftarrow \mathbb{F}^*, \tau = (\alpha, \beta, \gamma, \delta, x)$

Common Reference String(CRS):

$$\sigma = \left( \alpha, \beta, \gamma, \delta, \{x^i\}_{i=0}^{n-1}, \left\{ \frac{\beta u_i(x) + \alpha v_i(x) + w_i(x)}{\gamma} \right\}_{i=0}^{l}, \left\{ \frac{\beta u_i(x) + \alpha v_i(x) + w_i(x)}{\delta} \right\}_{i=l+1}^{m}, \left\{ \frac{x^i t(x)}{\delta} \right\}_{i=0}^{n-2} \right)$$

$\underline{\pi \leftarrow Prover:}$

Samples $r, s \leftarrow \mathbb{F}$ and creates a proof matrix $\Pi$

The proof is $\pi = \Pi\sigma = (A, B, C)\ where:$

$$A = \alpha + U(x) + r\delta, \qquad B = \beta + V(x) + s\delta,$$

$$C = \frac{\sum_{i=l+1}^{m} a_i \left( \beta u_i(x) + \alpha v_i(x) + w_i(x) \right) + h(x)t(x)}{\delta} + As + rB - rs\delta$$

# NILP for QAP

Verifier :

Checks if:

$$A \cdot B = \alpha \cdot \beta + \frac{\sum_{i=0}^{l} a_i \left(\beta u_i(x) + \alpha v_i(x) + w_i(x)\right)}{\gamma} \cdot \gamma + C \cdot \delta$$

The verifier accepts the proof if the equality holds

# Pairing-based NIZK

Encode field elements as group elements in bilinear groups

Split the CRS into two parts:

$$\sigma = ([\sigma_1]_1, [\sigma_2]_2)$$

Encode: $A, C$ in $G_1$ and $B$ in $G_2$: Also the proof becomes $\pi = (\pi_1, \pi_2)$

From $e: G_1 \times G_2 \rightarrow G_T$

$G_1$ side : $[\sigma_1]_1, [\pi_1]_1$

$G_2$ side : $[\sigma_2]_2, [\pi_2]_2$

Or more specifically the proof is written as:

$$\pi_1 = (\mathrm{A}, \mathrm{C}), \pi_2 = B \quad or \quad \pi = ([A]_1, [C]_1, [B]_2)$$

Then the verifier can check quadratic equations using pairings:

$$[\sigma_1, \pi_1]_1 \cdot T_i[\sigma_2, \pi_2]_2 = [0]_T$$

# Pairing-based NIZK Setup

Given the relation $R$ of the form:

$$R = (p, G_1, G_2, G_T, e, g, h, l, \{u_i(X), v_i(X), w_i(X)\}_{i=0}^{m}, t(X)),$$

$\underline{(\sigma, \tau) \leftarrow Setup:}$

Samples $\alpha, \beta, \gamma, \delta, x \leftarrow \mathbb{Z}_p^*, \tau = (\alpha, \beta, \gamma, \delta, x)$

Common Reference String (CRS): $\sigma = ([\sigma_1]_1, [\sigma_2]_2)$ *where:*

$$\sigma_1 = \left( \begin{array}{c} \alpha, \beta, \delta, \{x^i\}_{i=0}^{n-1}, \left\{\frac{\beta u_i(x) + \alpha v_i(x) + w_i(x)}{\gamma}\right\}_{i=0}^{l}, \\ \left\{\frac{\beta u_i(x) + \alpha v_i(x) + w_i(x)}{\delta}\right\}_{i=l+1}^{m}, \left\{\frac{x^i t(x)}{\delta}\right\}_{i=0}^{n-2} \end{array} \right)$$

$$\sigma_2 = \left(\beta, \gamma, \delta, \{x^i\}_{i=0}^{n-1}\right)$$

# Pairing-based NIZK Prover

$\underline{\pi \leftarrow Prover:}$

We want $\pi = ([A]_1, [C]_1, [B]_2)$

Samples $r, s \leftarrow \mathbb{Z}_p$ and creates a proof matrix $\Pi = \begin{pmatrix} \Pi_1 & 0 \\ 0 & \Pi_2 \end{pmatrix}$

The proof is $\pi = (\pi_1, \pi_2)$ where $\pi_1 = \Pi_1 \sigma_1 \; and \; \pi_2 = \Pi_2 \sigma_2$

$$[A]_1 = [\alpha]_1 + \sum_{i=0}^{m} a_i [u_i]_1 \; , + r[\delta]_1,$$

$$[B]_2 = [\beta]_2 + \sum_{i=0}^{m} a_i [v_i]_2 \; , + s[\delta]_2,$$

$$[C]_1 = \sum_{i=l+1}^{m} a_i \left[\frac{\left(\beta u_i(x) + \alpha v_i(x) + w_i(x)\right)}{\delta}\right]_1 + \sum_{i=0}^{n-2} h_i \left[\frac{x^i t(x)}{\delta}\right]_1 + [A]_1 s + r[B]_1 \; - rs[\delta]_1$$

# Pairing-based NIZK Verifier

$0/1 \leftarrow \boldsymbol{Verify}$:

Recall: $\pi = ([A]_1, [C]_1, [B]_2)$

Checks if:

$$[A]_1 \cdot [B]_2 = [\alpha]_1 \cdot [\beta]_2 + \sum_{i=0}^{l} a_i \left[\frac{\left(\beta u_i(x) + \alpha v_i(x) + w_i(x)\right)}{\gamma}\right]_1 \cdot [\gamma]_2 + [C]_1 \cdot [\delta]_2$$

The verifier accepts the proof if and only if the equality holds